

Paradigmas de Programación

Guía de Ejercicios

Stack, Heap y MIPS

Dr. Pablo Javier Vidal

Objetivo

El objetivo de esta guía es comprender cómo se organizan los datos en memoria durante la ejecución de un programa, en particular:

- organización de memoria del proceso (.text, .data, .bss, heap, stack)
- funcionamiento del stack frame (registro de activación)
- diferencia entre memoria automática (stack) y dinámica (heap)
- pasaje de parámetros
- modelo de ejecución en assembler MIPS

En cada ejercicio se deberá indicar:

- qué datos están en registros
- qué datos están en el stack
- qué datos están en el heap
- qué ocurre durante la llamada y retorno de funciones

Se recomienda utilizar MARS o QtSpim.

1. Parte A: Ejercicios de implementación en MIPS

Ejercicio 1

Escribir un programa en MIPS que:

- defina dos enteros en la sección .data
- los cargue en registros
- calcule la suma
- guarde el resultado en memoria

Indicar en qué región de memoria se encuentra cada dato.

Ejercicio 2

Escribir un programa que:

- reserve 4 bytes en el stack
- almacene el valor 8 en esa posición
- recupere el valor
- le sume 2
- guarde el resultado nuevamente en el stack

Indicar cómo cambia el valor del stack pointer.

Ejercicio 3

Implementar una función:

```
int doble(int x)
{
    return 2*x;
}
```

El programa principal debe llamar a la función y mostrar el resultado.

Indicar:

- en qué registro se pasa el parámetro
- en qué registro se devuelve el resultado

Ejercicio 4

Modificar el ejercicio anterior para que la función:

- reserve espacio en el stack para una variable local
- calcule $y = x + 3$
- retorne y

Indicar qué elementos forman el stack frame.

Ejercicio 5

Implementar dos funciones:

```
int inc(int x)
{
    return x+1;
}

int dobleMasUno(int x)
{
    return 2*x + 1;
}
```

La función dobleMasUno debe llamar a inc o a doble.

Indicar:

- cuándo es necesario guardar el registro \$ra en el stack
- cómo cambia el stack durante la llamada

Ejercicio 6

Reservar memoria dinámica en heap para almacenar un entero.

- guardar el valor 25
- leer el valor
- mostrarlo

Indicar:

- qué contiene el heap
- qué contiene el stack

Ejercicio 7

Implementar una función:

```
void poner30(int* dir)
{
    *dir = 30;
}
```

El programa debe:

- reservar memoria en heap
- inicializar el valor en 10
- llamar a la función
- verificar el cambio de valor

Ejercicio 8

Reservar espacio en heap para un vector de 3 enteros.

- almacenar los valores 10, 20 y 30
- sumar los valores
- guardar el resultado

Indicar cómo se calculan las direcciones de memoria de cada elemento.

Ejercicio 9

Escribir un programa que:

- reserve una variable local en stack con valor 7
- reserve memoria en heap
- copie el valor del stack al heap
- modifique el valor en heap a 50
- muestre ambos valores

Explicar por qué modificar el heap no modifica el valor original en stack.

Ejercicio 10

Implementar una función:

```
void cuadradoEnHeap(int x, int* dir)
{
    *dir = x*x;
}
```

El programa debe:

- reservar memoria en heap
- llamar a la función
- verificar el resultado

Explicar el flujo de datos entre registros, stack y heap.

2. Parte B: Ejercicios teóricos

Ejercicio 11

¿Es necesario guardar siempre el registro \$ra en el stack? Justificar.

Ejercicio 12

Describir los elementos que forman parte de un stack frame típico.

Ejercicio 13

Explicar qué ocurre en memoria cuando una función recursiva no alcanza el caso base.

Ejercicio 14

Comparar:

- variable local en stack
- variable dinámica en heap

Analizar diferencias en tiempo de vida y forma de acceso.

Ejercicio 15

¿Por qué en MIPS no se pueden sumar directamente dos posiciones de memoria?

Ejercicio 16

Explicar por qué el stack modela naturalmente llamadas anidadas.

Ejercicio 17

Si una función guarda una dirección de heap en un registro temporal y luego llama a otra función, ¿qué problema puede ocurrir?

Ejercicio 18

¿Por qué puede haber múltiples stack frames de una misma función?

Ejercicio 19

¿Qué diferencia existe entre:

- jerarquía de bloques
- árbol de activación

Ejercicio 20

Explicar la relación entre:

- stack pointer
- frame pointer
- program counter

3. Parte C: Definiciones

Definir brevemente los siguientes conceptos:

1. Stack
2. Heap
3. Registro de activación (stack frame)
4. Stack pointer
5. Dirección de retorno
6. Arquitectura load/store
7. Parámetro por valor
8. Parámetro por referencia
9. Variable local
10. Región .data
11. Región .text
12. Memoria dinámica
13. LIFO
14. Registro
15. Dirección de memoria

Pregunta integradora

Explicar cómo se relacionan:

- llamadas a funciones
- stack frame
- heap
- registros
- ejecución del programa

Se recomienda incluir un esquema conceptual.